



Diana Carrera – Leonel Tipan – Paul Moreno
GRUPO 7

Departamento de Ciencias de la Computación
de Software, Universidad de las Fuerzas Armadas ESPE,
Av. General Rumiñahui s/n Sangolquí-Ecuador

TEMA: Análisis de trazabilidad
Casos de uso, clases y secuencia

MATERIA: Análisis y Diseño de
Software

NRC: 30723

1. Casos de Uso:

1.1. Código:

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false

actor "Administrador académico" as Admin
actor "Sistema" as Sistema

rectangle "Sistema CRUD de Estudiantes" {
    usecase "Agregar estudiante" as UC1
    usecase "Actualizar estudiante" as UC2
    usecase "Eliminar estudiante" as UC3
    usecase "Mostrar todo" as UC4
    usecase "Validar datos del estudiante" as UC5
}

Admin --> UC1
Admin --> UC2
Admin --> UC3
Admin --> UC4

UC1 ..> UC5 : <<include>>
UC2 ..> UC5 : <<include>>

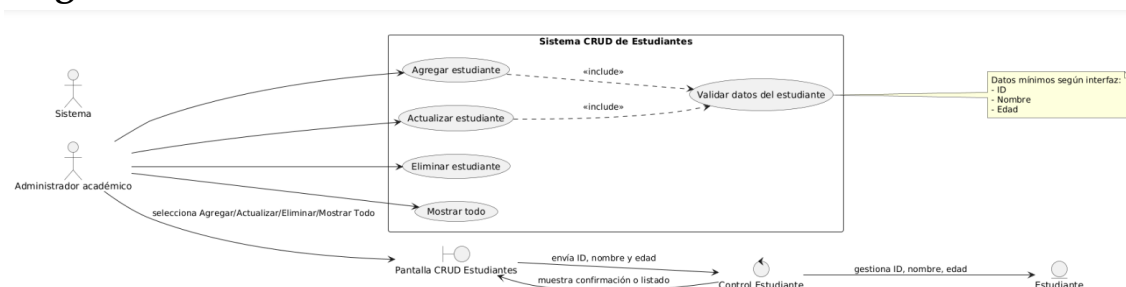
note right of UC5
Datos mínimos según interfaz:
- ID
- Nombre
- Edad
end note
@enduml
```

```
@startuml
left to right direction

actor "Administrador académico" as Admin
boundary "Pantalla CRUD Estudiantes" as UI
control "Control Estudiante" as Ctrl
entity "Estudiante" as Est

Admin --> UI : selecciona Agregar/Actualizar/Eliminar/Mostrar Todo
UI --> Ctrl : envía ID, nombre y edad
Ctrl --> Est : gestiona ID, nombre, edad
Ctrl --> UI : muestra confirmación o listado
@enduml
```

1.2. Imagen:



2. Clases de Análisis

2.1. Código

```
@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false
left to right direction

class "FormularioCrudEstudiante" <<boundary>> {
    - txtId: String
    - txtNombre: String
    - txtEdad: String
    + clickAgregar()
    + clickActualizar()
    + clickEliminar()
    + clickMostrarTodo()
    + mostrarMensaje(mensaje: String)
    + mostrarTabla(estudiantes: List<Estudiante>)
}

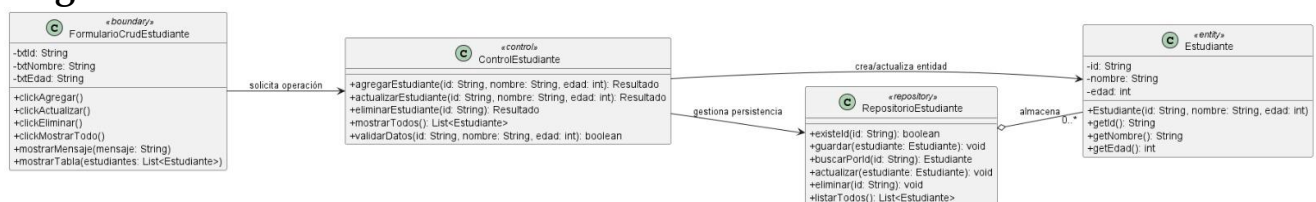
class "ControlEstudiante" <<control>> {
    + agregarEstudiante(id: String, nombre: String, edad: int): Resultado
    + actualizarEstudiante(id: String, nombre: String, edad: int): Resultado
    + eliminarEstudiante(id: String): Resultado
    + mostrarTodos(): List<Estudiante>
    + validarDatos(id: String, nombre: String, edad: int): boolean
}

class "Estudiante" <<entity>> {
    - id: String
    - nombre: String
    - edad: int
    + Estudiante(id: String, nombre: String, edad: int)
    + getId(): String
    + getNombre(): String
    + getEdad(): int
}

class "RepositorioEstudiante" <<repository>> {
    + existeId(id: String): boolean
    + guardar(estudiante: Estudiante): void
    + buscarPorId(id: String): Estudiante
    + actualizar(estudiante: Estudiante): void
    + eliminar(id: String): void
    + listarTodos(): List<Estudiante>
}

FormularioCrudEstudiante --> ControlEstudiante : solicita operación
ControlEstudiante --> Estudiante : crea/actualiza entidad
ControlEstudiante --> RepositorioEstudiante : gestiona persistencia
RepositorioEstudiante o-- "0..*" Estudiante : almacena
@enduml
```

2.2. Imagen



3. Secuencias de Análisis

3.1. Secuencia agregar estudiante

- Código:

```

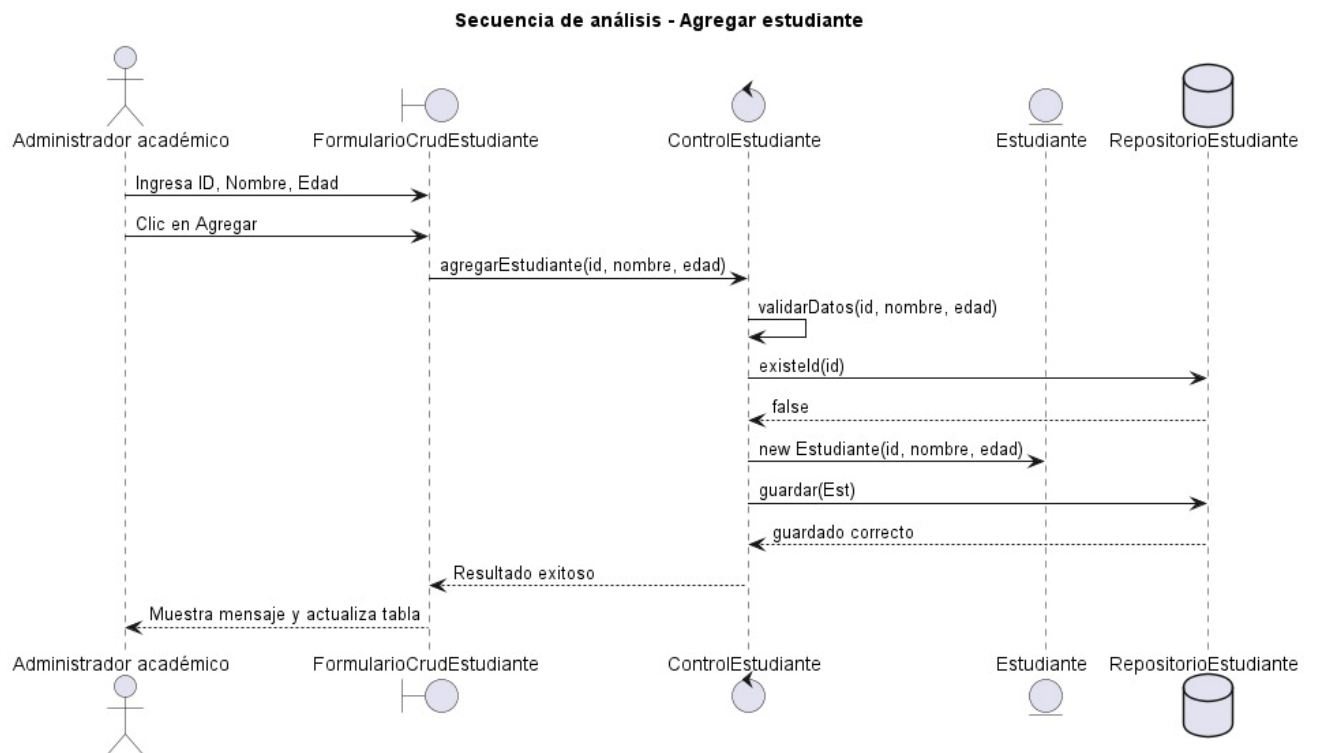
@startuml
title Secuencia de análisis - Agregar estudiante
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
entity "Estudiante" as Est
database "RepositorioEstudiante" as Repo

Admin -> UI : Ingresa ID, Nombre, Edad
Admin -> UI : Clic en Agregar
UI -> Ctrl : agregarEstudiante(id, nombre, edad)
Ctrl -> Ctrl : validarDatos(id, nombre, edad)
Ctrl -> Repo : existeId(id)
Repo --> Ctrl : false
Ctrl -> Est : new Estudiante(id, nombre, edad)
Ctrl -> Repo : guardar(Est)
Repo --> Ctrl : guardado correcto
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje y actualiza tabla
@enduml

```

■

■ Imagen



■

3.2. Secuencia Actualizar estudiante

■ Código:

```

@startuml
title Secuencia de análisis - Actualizar estudiante
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
database "RepositorioEstudiante" as Repo
entity "Estudiante" as Est

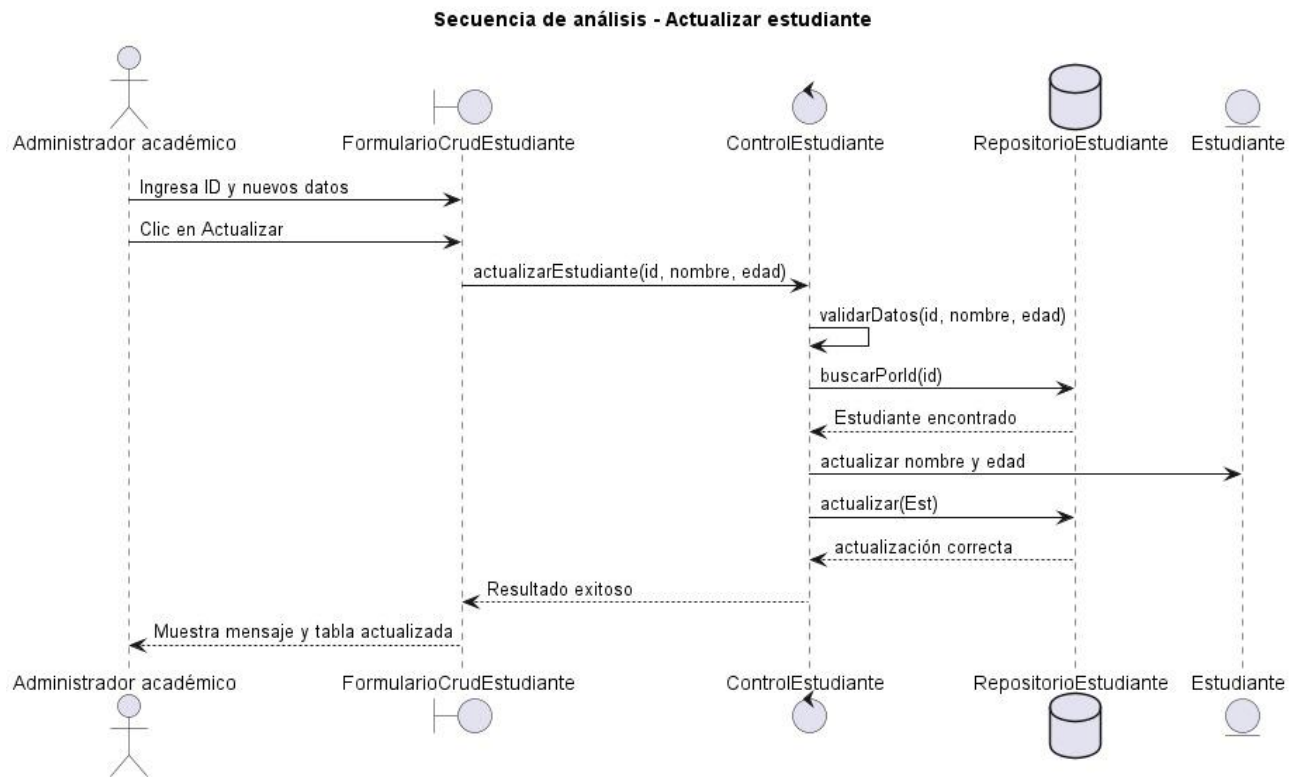
Admin -> UI : Ingresa ID y nuevos datos
Admin -> UI : Clic en Actualizar
UI -> Ctrl : actualizarEstudiante(id, nombre, edad)
Ctrl -> Ctrl : validarDatos(id, nombre, edad)
Ctrl -> Repo : buscarPorId(id)
Repo --> Ctrl : Estudiante encontrado
Ctrl -> Est : actualizar nombre y edad
Ctrl -> Repo : actualizar(Est)
Repo --> Ctrl : actualización correcta

```

```
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje y tabla actualizada
@enduml
```

■

■ Imagen



■

3.3. Secuencia Eliminar estudiante

■ Codigo

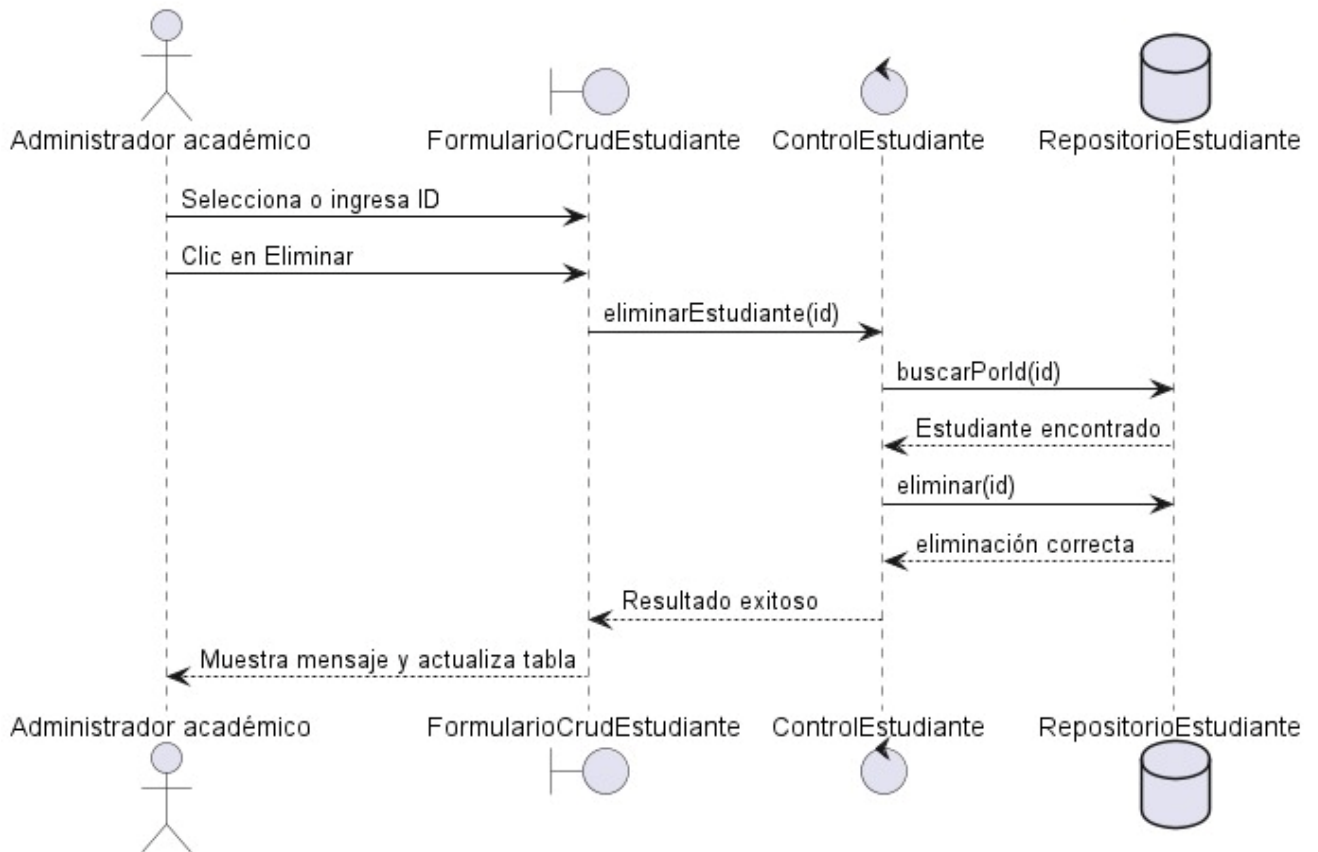
```
@startuml
title Secuencia de análisis - Eliminar estudiante
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
database "RepositorioEstudiante" as Repo

Admin -> UI : Selecciona o ingresa ID
Admin -> UI : Clic en Eliminar
UI -> Ctrl : eliminarEstudiante(id)
Ctrl -> Repo : buscarPorId(id)
Repo --> Ctrl : Estudiante encontrado
Ctrl -> Repo : eliminar(id)
Repo --> Ctrl : eliminación correcta
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje y actualiza tabla
@enduml
```

■

■ Imagen

Secuencia de análisis - Eliminar estudiante



3.4. Secuencia mostrar todos los estudiantes

■ Código

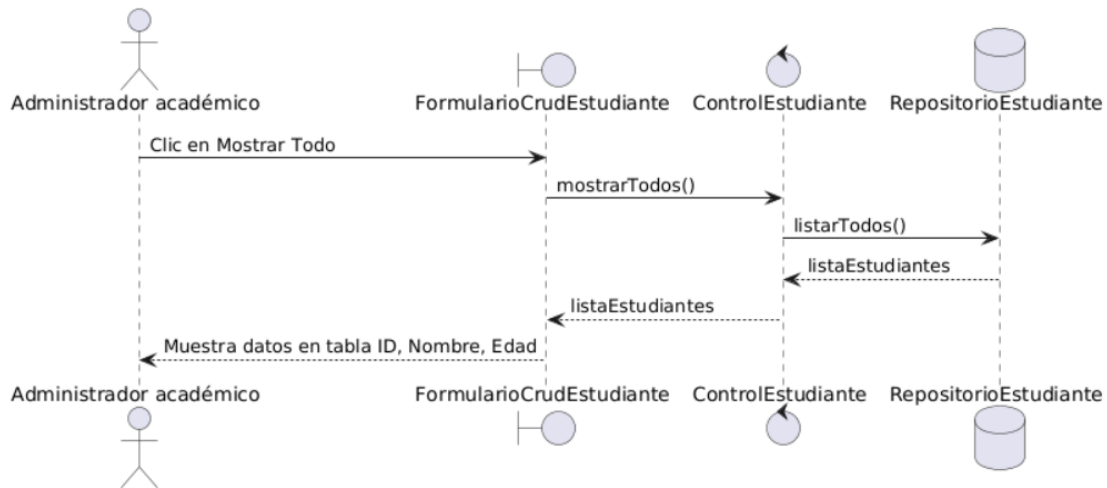
```

@startuml
title Secuencia de análisis - Mostrar todos los estudiantes
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
database "RepositorioEstudiante" as Repo

Admin -> UI : Clic en Mostrar Todo
UI -> Ctrl : mostrarTodos()
Ctrl -> Repo : listarTodos()
Repo --> Ctrl : listaEstudiantes
Ctrl --> UI : listaEstudiantes
UI --> Admin : Muestra datos en tabla ID, Nombre, Edad
@enduml
    
```

■ Imagen

Secuencia de análisis - Mostrar todos los estudiantes



4. Trazabilidad

RF	Requisito funcional	Caso de uso asociado	Clases de análisis relacionadas	Secuencia de análisis
RF-01	El sistema debe permitir agregar un nuevo estudiante con ID, Nombre y Edad.	Agregar estudiante	FormularioCrudEstudiante<<boundary>> ControlEstudiante<<control>> Estudiante<<entity>> RepositorioEstudiante<repository>>	Secuencia: Agregar estudiante
RF-02	El sistema debe permitir actualizar los datos de un estudiante existente usando ID, Nombre y Edad.	Actualizar estudiante	FormularioCrudEstudiante<<boundary>> ControlEstudiante<<control>> Estudiante<<entity>> RepositorioEstudiante<repository>>	Secuencia: Actualizar estudiante
RF-03	El sistema debe permitir eliminar un estudiante seleccionado cuando corresponda según la regla académica definida.	Eliminar estudiante	FormularioCrudEstudiante<<boundary>> ControlEstudiante<<control>> Estudiante<<entity>> RepositorioEstudiante<repository>>	Secuencia: eliminar estudiante
RF-04	El sistema debe permitir mostrar todos los estudiantes registrados en una tabla con las columnas ID, Nombre y Edad.	Mostrar todo	FormularioCrudEstudiante<<boundary>> ControlEstudiante<<control>> Estudiante<<entity>> RepositorioEstudiante<repository>>	Secuencia Mostrar todos los estudiantes
RF-05	El sistema debe validar los datos obligatorios antes de guardar o actualizar cambios.	Validar datos del estudiante	FormularioCrudEstudiante<<boundary>> ControlEstudiante<<control>> Estudiante<<entity>>	Se integra dentro de las secuencias: agregar estudiante y actualizar estudiante

5. Conclusión técnica

Los requisitos funcionales identificados en el ejercicio CRUD de estudiantes van como las necesidades que necesita cubrir el sistema, de esta manera la implementación formal del requisito funcional sería el caso de uso donde identificamos el actor que realiza la acción, las precondiciones, postcondiciones y proceso a realizarse; a partir de ello podemos ver cuáles son los conceptos que identificamos para el proceso de implementación que se traducen en las clases con los atributos y métodos, y finalmente para entender cómo los atributos y los métodos se relacionan, los actores como desencadenantes de este proceso y cómo los datos fluyen a través de las clases. La trazabilidad entre RF, CU, diagrama de clases y diagrama de secuencia se da de manera que se muestra un panorama completo de cómo funciona el dominio del negocio (problema), estructura, parámetros a tomar en cuenta y el proceso a seguir. Su objetivo es principalmente formular de manera clara el proceso de trabajo.